NP-Hard and NP-Complete problems:

* Basic concepts
* Non Deterministic Algorithms.
* NP-Hard and NP-complete classes
* COOK's theorem.

Basic concepts:

→ Some computational problems are complex and difficult to find efficient algorithms for solving them and it cannot be solved in future also and have non polynomial time algorithm and some problems are there, that can be solved by a time algorithm, like $O(n)$, $O(\log n)$, $O(n \log n)$ and $O(n^2)$. polynomial time algorithms are being tractable and problems that require non-polynomial time as being ~~tract~~ intractable.

→ Any problem for which answer is either yes or no is called decision problem. The algorithm for decision problem is called decision algorithm.

→ Any problem that involves the identification of optimal cost (minimum or maximum) is called optimization problem. The algorithm for optimization problem is called optimization algorithm.

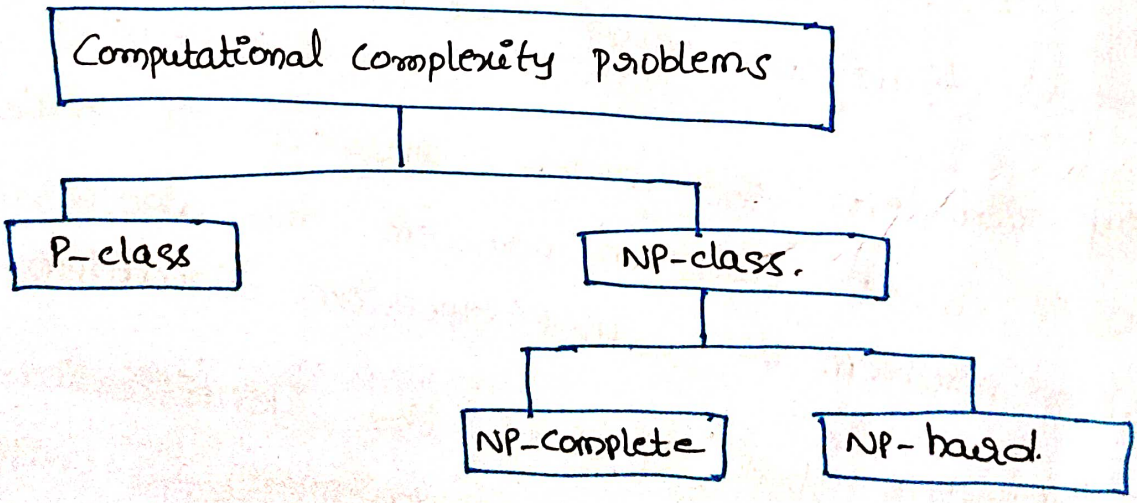
Definition of P: problems that can be solved in polynomial time. ("P" stands for polynomial).

Examples: Seaaching of key element, sorting of elements, All pair shortest path.

Definition of NP: It stands for "non-deterministic Polynomial time". Note that NP does not stand for "non-polynomial".

Examples: Travelling salesperson problem, Graph coloring problem, Knapsack problem, Hamiltonian circuit problems.

→ The NP-class problems can be further categorized into NP-complete and NP-hard problems



classification of Complexity problems

→ The theory of NP-completeness problem, whose status is unknown, does not provide a method for obtaining polynomial time algorithms for the problems whose computing times are greater than polynomial but, what we do is show that many problems for which they are no known polynomial time algorithms are computationally related.

→ here we establish two classes of problems. They are

1) NP-Hard

2) NP-complete

→ A problem that is NP-complete has the property that it can be solved in polynomial time iff all other NP-complete problems can also be solved in polynomial time.

→ If an NP-hard problem can be solved in polynomial time, then all NP-complete problems can be solved in polynomial time.

→ All NP-complete problems are NP-hard but some NP-hard problems are not known to be NP-complete

Non-deterministic algorithms

→ The algorithm in which every operation is uniquely defined is called deterministic algorithm.

→ The algorithm in which the operations are not uniquely defined but are limited to specified set of possibilities for every operation, such an algorithm is called non deterministic algorithm.

→ The non-determinstic algorithms use the following functions.

1. choice: Arbitrarily chooses one of the element from given set.

2. Failure: Indicates an unsuccessful completion.

3. success: Indicates a successful completion.

→ A non deterministic algorithm terminates unsuccessfully if and only if there exists no set of choices leading to a success signal.

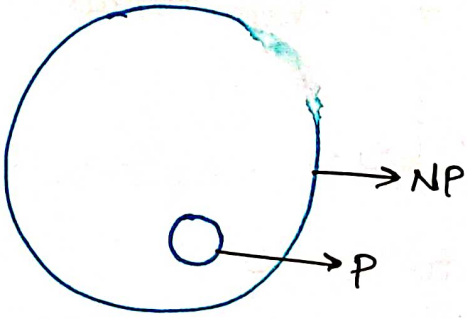
→ Whenever there is a set of choices that leads to a successful completion, then one such set of choices is selected and the algorithm terminates successfully.

→ In case the successful completion is not possible then the time complexity is O(1).

→ In case of successful signal completion then the time required is the minimum no. of steps needed to reach a successful completion of (n). where n is the no. of inputs.

⇒ A machine capable of executing non-deterministic algorithms in this way is called a <u>non-deterministic machine</u>.

⇒ A Non deterministic algorithm is considered as a superset for deterministic algorithm i.e. the deterministic algorithms are a special case of non-determinstic algorithms.



<u>Example 1</u>: A non deterministic algorithm for searching an element x in the given set of elements A[1:n] in the time complexity O(1).

```
Algorithm NonDetSearch(A,n)
{
    K:=choice(1,n);
    if A[k] = x then
    {
        write(K);
        Success();
    }
    write(0);
    Failure();
}
```

Example 2: Non-deterministic algorithm for sorting.

→ Let A[1:n] be an unsorted array of positive integers B[1:n] is an auxillary array. The Non-deterministic algorithm NDSORT (A, n) sorts the numbers into non-decreasing order.

```
Algorithm. NonDetSort (A, n)
// This is a non-deterministic sorting algorithm
{
    for j=1 to n do
        s[j]:=0;
    for j:=1 to n do
    {
        k:= choice(1, n);
        if s[k] ≠ 0 then failure();
        else
        s[k] = A[j];
    }
    for j=1 to n-1 do
        if s[j] > s[j+1] then Failure();
    write (s[1:n]);
    success();
}
```

Example 3:

Decision problem:

Definition: Any problem for which the answer is either 0 or 1 is called a decision problem. The corresponding for a decision problem is called decision algorithm.

→ An optimization problem is one that identifies an optimal value of a given function (either a maximum value or a minimum value). The algorithm associated with an optimization problem is called an optimization algorithm.

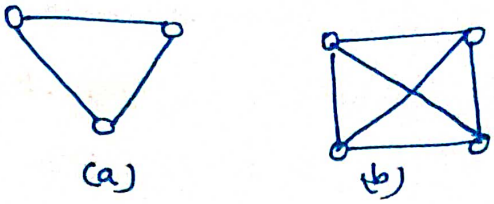
Maximum clique: A maximal complete subgraph of a graph $G = (V, E)$ is a clique, which is isomorphic to K_n, where K_n is complete graph on n vertices.

→ The size of the clique is the no. of vertices in it.

→ Maximum clique problem is an optimization problem that has to determine the size of a largest clique in G. we make use of decision problem to determine whether G has a clique of size atleast k for some given k.

→ Let us consider a D clique (G, K) be the deterministic decision algorithm for the clique decision problem.

→ Let n be the no. of vertices in G. The size of max clique in G can be found by making several applications of D clique. D clique is used once for each K, $K = n, n-1, n-2 \ldots$, until the o/p from D clique is 1.



Complete Graphs

→ If the time complexity of D clique is $f(n)$ then the size of most clique can be found in time $\leq n\, f(n)$. Also, if the size of a maximum clique can be determined in time $g(n)$, then the decision problem can be solved in time $g(n)$.

→ Hence the maximum clique problem is solved in polynomial time iff the clique decision problem can be solved in polynomial time.

⇒ The input to the maximum clique decision problem can be provided as a sequence of edges and an integer K. Each edge in E(G) is a pair of numbers (i, j) the size of the input for each edge (i, j) is $\log_2^i + \log_2^j + 2$ if a binary representation is assumed. The input size of any instance, is.

$$n = \sum_{\substack{(i,j) \in E(G) \\ i<j}} \left(\left[\log_2^i\right] + \left[\log_2^j\right] + 2 \right) + \log_2^K + 1$$

→ If G has only one connected component then $n \geq |V|$. Thus if this decision problem cannot be solved by an algorithm of complexity $P(n)$ for some $P()$, then it cannot be solved by an algorithm of complexity $P(|V|)$.

→ Algorithm DCK is a non-deterministic algorithm for the clique decision problem. Algorithm DCK begins to form set of K distinct vertices.

→ Then it tests to see whether these vertices form a complete subgraph. If G is given by its adjacency matrix and |V|=n, the input length m is $n^2 + \log_2^k + \log_2^n + 2$. The for loop of lines 4 to 9 can easily be implemented to run in nondeterministic time O(n). The time for the loop 11 and 12 is $O(k^2)$. Hence the overall nondeterministic time is $O(n+k^2) = O(n^2)$ = O(m). There is no known polynomial time deterministic algorithm for this problem.

```
Algorithm DCK (G, n, K)
{
    S = φ;
    for i=1 to K do
    {
        t = choice(1, n);
        if t∈S then failure();
    } S = S∪{t}     // combine t to set S
    // Here S contains K distinct vertex indices
    for all pairs (i, j)
    such that i∈S, j∈S and i≠j do if(i,j) is not
    an edge of G
    then failure();
    success();
}
```

Example 4: Satisfiability problem

→ We formulate satisfiability problem to determine whether a formula is true for some assignment of truth values to the variables. The boolean formula ϕ is composed of,

1) Boolean variables $x_1, x_2 \ldots$

2) Boolean connectives or any bodean functions with one or two inputs and one output such as A (AND), V (OR), ~ (NOT), → (implication), ↔ (if and only if) and

3) Parenthesis

→ A truth assignment for a boolean formula ϕ is a set of values for variables of ϕ, and a satisfying assignment is a truth assignment that causes it to evaluate to 1. A formula with a satisfying assignment is a satisfiable formula.

CNF (Conjunctive Normal Form): A formula is in CNF iff it is represented as $\bigwedge_{i=1}^{k} C_i$ as where the C_i are clauses each represented as $\bigvee I_{ij}$ I_{ij} are literals.

DNF (Disjunctive Normal Form): It is in DNF iff it is represented as $\bigvee_{i=1}^{k} C_j$ and each clause C_i is represented as $\bigwedge I_{ij}$ $(x_1 \wedge x_2) \vee (x_3 \wedge \bar{x}_4)$ is in DNF whereas $(x_1 \vee \bar{x}_4) \wedge (x_1 \vee \bar{x}_2)$ is in CNF.

NP- Hard and NP- complete classes:

- The classes NP-Hard and NP- complete
- Halting problem

→ An algorithm is said to be "polynomially bounded" if its worst-case complexity is bounded by a polynomial function of the input size (i.e., if there is a polynomial 'p' such that for each input of size 'n' the algorithm terminates after at most P(n) steps.

→ A problem is said to be polynomially bounded if there is a polynomially bounded algorithm for it.

→ "P" is the class of decision problems that are polynomially bounded. "P" is defined only for decision problems.

→ "NP" is the class of decision problems for which there is polynomially bounded nondeterministic algorithm.

NP- complete problems:

⇒ A problem L is NP- complete iff L is NP- Hard and L ∈ NP (non-deterministic polynomial).

⇒ A problem that is NP- complete has the property that it can be solved in polynomial time iff all other NP-complete problems can also be solved in polynomial time

⇒ If any NP-hard problem can be solved in polynomial time, then all NP- complete problems can be solved in polynomial time. All NP- complete problems are NP hard, but some NP- hard problems are

not known to be NP-complete. Normally the decision problems are NP-complete but optimization problems are NP-Hard

→ However if problem L_1 is a decision problem and L_2 is an optimization problem then it is possible that $L_1 \propto L_2$ For example the Knapsack decision problem can be reduced Knapsack optimization problem. There are some NP hard problems that are not NP complete

* **NP-Hard:**

A problem L is NP-Hard iff satisfiability reduces to L i.e, Any non-deterministic polynomial time problem is satisfiable and reducable then the problem is said to be NP-Hard.

Ex: Halting problem, Flow-shop scheduling.

* CLASSES of NP-HARD and NP-COMPLETE PROBLEMS:

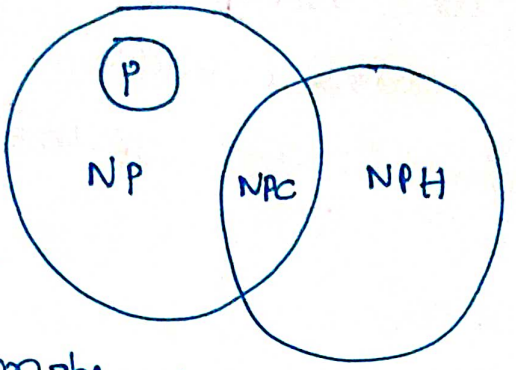
→ Let P, NP, NPH and NPC are the sets of all possible decision problems that are solvable in polynomial time by using deterministic algorithms, non deterministic algorithms, NP-Hard and NP-complete respectively.

→ then the relationship b/w P, NP, NPH and NPC can be expressed using venn diagrams is presented below.

1.) Most theoretical computer scientists view the relationships among P, NP, NPC as both P and NPC are wholly contained within NP and $P \cap NPC = \phi$,

$NPC = NP = NP \cap NPH$.

2. When NP = NPH and P = NP ∩ NPH then the relationship between P, NP, NPC and NPH is



Relationship between P, NP and NPC and NPH

Halting problem: Halting problem is an example of an NP-hard decision problem that is not NP-complete.

→ The halting problem is to determine for an arbitrary deterministic algorithm A and an i/p I whether algorithm A with input I ever terminates. (or enters an infinite loop.

→ This problem is undecidable i.e, there exists no algorithm to solve this problem. so it clearly cannot be in NP.

→ To show satisfiability of halting problem, simply construct an algorithm A whose input is a propositional formula x. If x has n variables then A tries out all 2^n possible truth values and verifies if x is satisfiable. If it is then A stops. If x is not satisfiable then A enters an infinite loop. Hence, A halts on input x

iff X is satisfiable.

→ if we had a polynomial time algorithm for the halting problem then we should solve the satisfiability problem in polynomial time using A and X as i/p to the algorithm for the halting problem.

→ Hence, the halting problem is an NP-hard problem which is not in NP.

Cook's Theorem:

Cook's Theorem states that satisfiability is in P. iff P=NP. we already know that satisfiability is in NP. Hence if P=NP, then satisfiability is in P.

⇒ Then it remains to be shown that if satisfiability is in P then P=NP. In order to prove this statement, we show how to obtain any polynomial time non deterministic decision algorithm A and input I, a formula Q(A, I) such that Q is satisfiable if A has successful termination with input I. If the length of I is 'n' and the time complexity of A is P(n) for some polynomial P() then the length of Q will be $O(P^3(n) \log n) = O(P^4(n))$. The time needed to construct Q will also be same.

> A deterministic algorithm Z to determine the outcome of A on any input I may be obtained as

If (Z) computes Q and then uses a deterministic algorithm for the satisfiability problem to determine whether or not Q is satisfiable. If $O(q(m))$ is the time needed to determine if a formula of length 'm' is satisfiable then the complexity of Z is ~~$O(p(n)\log n +$~~ $O(p^3(n)\log n + q(p^3(n)\log n)$. If satisfiability is in P then $q(m)$ is a polynomial function of m and the complexity of Z becomes $O(r(n))$ for some polynomial $r()$.

→ Hence, if satisfiability is in P then for every non deterministic algorithm A in NP we can obtain a deterministic Z in P.

⇒ Assumption for constructing the boolean formula Q for a given non-deterministic algorithm A:

1. A is to be executed in word oriented.
2. A simple expression is an expression that contains one operator and all operands are simple variables.
   (i) (simple variable ← simple expression)
   (ii) (array variable) ← (simple variable)
   (iii) (simple variable) ← (array variable)
   (iv) (simple variable) ← (choice of S)
3. All variables in A are of type integer or boolean.
4. Algorithm A contains no read or write statements
5. Algorithm A contains no constants.

6. In addition to simple assignment statement A is allowed to contain only the following type of statements

(i) statement goto K

(ii) Statement if(L) then goto b

(iii) Success(), failure()

7. Algorithm A contain type declaration and dimension statements

8. Let P(n) be a polynomial time such that A takes no more than P(n) time on the input of length n.